

task_bridge

Dein Aufgabenboard und deine Single Source of Truth - makellos synchronisiert, in beide Richtungen.

Zusammenfassung

task_bridge ist eine generische, entkoppelte, bidirektionale Synchronisations-Engine für Aufgaben/Boards in Go. Sie hält die **Single Source of Truth** der bearbeitbaren Elemente (SQLite) eines Projekts synchron mit dessen Tracker-Dokumenten und einem Remote-Board (erster Ziel: ClickUp; Jira/Linear geplant) – nach dem Prinzip „letzte Änderung gewinnt“, mit Dry-Run-Vorabprüfung und garantierter Datenintegrität.

Kurzbeschreibung

Ein projektunabhängiges Go-Submodul, das bidirektional eine **Single Source of Truth** für bearbeitbare Elemente (SQLite) ↔ Tracker-Dokumente ↔ ein Remote-Board (zunächst ClickUp) synchronisiert. Deterministisch nach dem „letzte Änderung gewinnt“-Prinzip, mit Dry-Run-Vorabprüfung und HMAC-gesicherten Webhooks; alle Anmeldedaten und IDs werden zur Laufzeit vom Nutzer injiziert.

Ausführliche Beschreibung

Jedes Team führt irgendwann zwei Bücher über dieselbe Arbeit: das echte – Code, Dokumentation, eine interne Datenbank – und das, das Manager im Blick haben, ein Board wie ClickUp. Die beiden laufen auseinander, sobald eine der Seiten bearbeitet wird, und sie manuell abzugleichen, ist genau die Art von langweiliger, fehleranfälliger Arbeit, die niemand zuverlässig erledigt. task_bridge wurde entwickelt, um diese Lücke zu schließen, indem alle drei Repräsentationen als ein System behandelt werden, das im Gleichschritt gehalten wird: die **Single Source of**

Truth der bearbeitbaren Elemente (SQLite) eines Projekts, seine **Tracker-Dokumentation** und ein **Remote-Board** – zunächst ClickUp, mit Jira und Linear als zukünftigen Optionen. Die Synchronisation ist deterministisch („letzte Änderung gewinnt“), erfolgt zunächst im Dry-Run-Modus und ist um ein einziges, unverhandelbares Versprechen herum konstruiert: Sie wird niemals Daten beschädigen oder verlieren und niemals eine Seite unbemerkt veralten lassen. In einem Bereich, in dem eine unachtsame Synchronisation eine Woche Arbeit überschreiben kann, ist diese Sicherheitsphilosophie der entscheidende Punkt.

Architektonisch ist es ein striktes Submodul, das von anderen Projekten genutzt wird und gemäß dem Entkopplungsvertrag der Verfassung (§11.4.28) vollständig projektunabhängig ist: Es liefert keine projektspezifischen Werte aus, und alle Anmeldedaten, Board-/Ordner-IDs, Feldnamen für Elementschlüssel und Datenbankpfade werden zur Laufzeit vom Nutzer über pkg/config.Config injiziert. Das Modul ist klar geschichtet: eine CLI-Schnittstelle (reconcile/push/pull/resolve/status/conflicts/init) und ein langlaufender Daemon (Webhook-Empfänger + Cron-basierte Synchronisation); ein schlanker Client-Wrapper über die MIT-lizenzierte Bibliothek raksul/go-clickup; ein Resolver, der Board-/Ordner-URLs via Live-Probes (API) in IDs umwandelt (ohne URL-Grammatik-Raten); ein Mapper zwischen lokalen bearbeitbaren Elementen und Remote-Aufgabenfeldern; eine Synchronisations-Engine nach dem „letzte Änderung gewinnt“-Prinzip mit expliziten Konfliktlösungen; sowie ein Webhook-Empfänger, der x-Signature-HMAC-SHA256 verifiziert. Es ist transparent in Bezug auf den Reifegrad: Dies ist das P1-Grundgerüst – Aufbau, Schnittstellen, Einstiegspunkte und die Entkopplungsgrenze sind vorhanden, doch die Synchronisationslogik und Live-Aufrufe zu ClickUp sind noch nicht implementiert (jeder Stub gibt gemäß der „No-Fakes“-Regel einen expliziten *Not-Implemented*-Fehler zurück).

Warum wir es entwickelt haben

Teams halten den „echten“ Stand der Arbeit in Code und Dokumentation fest, während Manager in einem Board wie ClickUp arbeiten – und die beiden laufen ständig auseinander. `task_bridge` macht sie zu einem System, das deterministisch und sicher synchronisiert, sodass keine Seite veraltet oder falsch wird.

Warum es ein Game-Changer ist

Zweiseitige Board-Synchronisation ist normalerweise eine einmalige, fest verdrahtete Integration, die jedes Team schlecht neu aufbaut. **task_bridge** definiert sie neu als

wiederverwendbare, mit Anmeldedaten injizierte Bibliothek mit strengen Datensicherheitsgarantien – Dry-Run zuerst, deterministische „Letzte-Änderung-gewinnt“-Logik, HMAC-geprüfte Ereignisse – sodass jedes Projekt vertrauenswürdige Board-Integration durch Konfigurationseinschleusung nutzen kann, statt erneut einen fragilen Konnektor zu schreiben, der an seine internen Strukturen gekoppelt ist.

Was innovativ ist

- Dreiseitige bidirektionale Synchronisation: **SQLite** SSoT ↔ Tracker-Dokumente ↔ Remote-Board.
- Vollständige Entkopplung (§11.4.28): keine projektspezifischen Werte; alles wird zur Laufzeit injiziert.
- Live-**API-URL**→**ID**-Auflösung statt fragiler **URL**-Grammatik-Parsing.
- HMAC-SHA256-geprüfte Webhook-Verarbeitung für Live-Ereignisse.

Herausforderungen & Lösungen

- **Datensicherheit über drei Quellen hinweg:** gelöst durch deterministische „Letzte-Änderung-gewinnt“-Logik, Dry-Run zuerst und explizite Konfliktauflösungen.
- **Wiederverwendbarkeit ohne Kopplung:** gelöst über die pkg/config-Injektionsgrenze (keine projektspezifischen Daten im Lieferumfang).
- **Zuverlässige Board-Identifikation:** gelöst durch URL-zu-ID-Auflösung mittels Live-**API**-Abfragen.
- **Ehrliches Scaffolding:** gelöst durch explizite „Nicht-implementiert“-Fehler bei nicht umgesetzten Stubs (keine Platzhalter).

Technologie-Stack (Warum + Wie)

- **Go** – Engine, **CLI** (cmd/task_bridge) und Daemon (cmd/task_bridged).
- **SQLite** – die arbeitsfähige Single Source of Truth für Elemente.
- **raksul/go-clickup (MIT)** – ClickUp-Transport-Wrapper.
- **HMAC-SHA256** – Webhook-Signaturprüfung.
- **cron + Webhooks** – Daemon-Abgleich + Live-Ereignisverarbeitung.
- **pkg/config** – Laufzeit-Injektionsgrenze für Anmeldedaten/IDs.

Status-Transparenz: Dies ist ein **P1-Scaffold** – die Synchronisationslogik ist noch nicht implementiert. Nicht als ausgeliefert präsentieren.